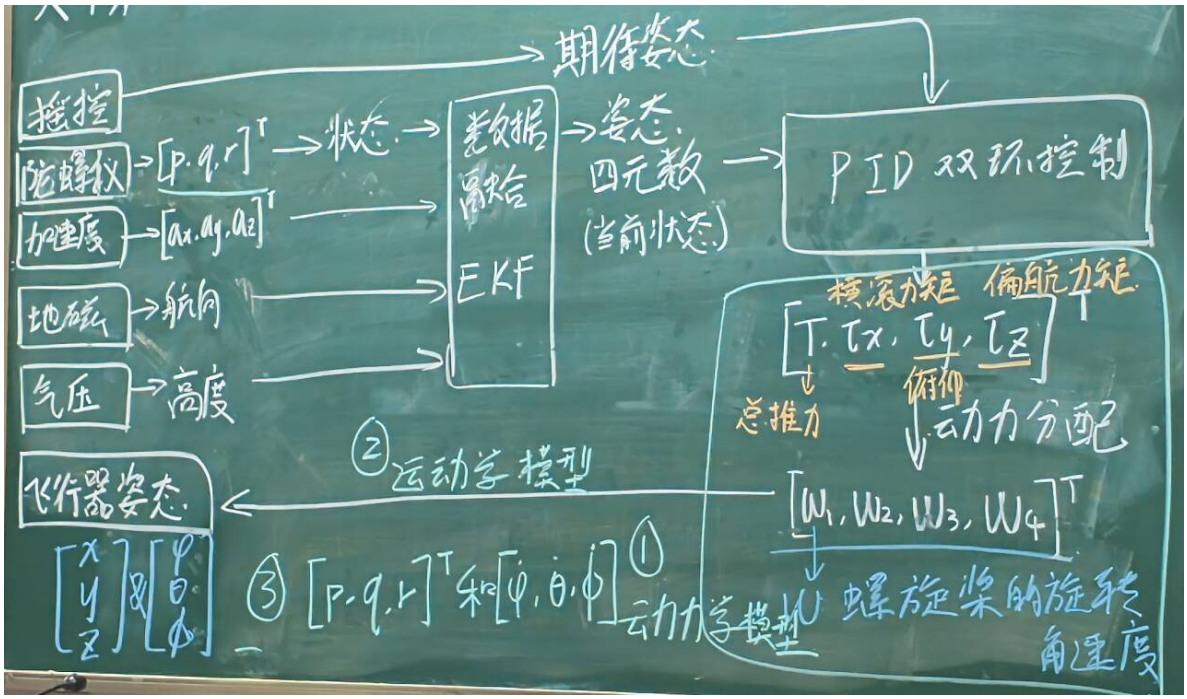
PID



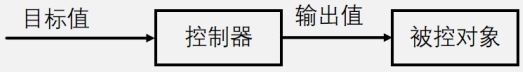
什么是PID?

1. 从一个控制系统开始：
    - 控制系统的基本思想：

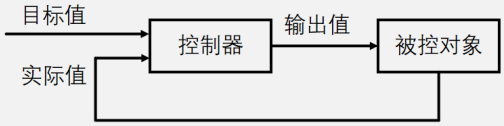
      找出如何产生合适的控制信号（输入）来使我们的系统产生所需的被控变量（输出）
    - 开环控制系统：

      > 缺点：只能大概的控制对象，不能精准的控制物理量的具体值，更不能根据实际情况动态调整输出值

      - 开环（Open Loop）：控制器单向输出值给被控对象，不获取被控对象的反馈，控制器对被控对象的执行状态不清楚

    - 闭环控制系统：
      - 闭环（Closed Loop）：控制器输出值给被控对象，同时获取被控对象的反馈，控制器知道被控对象的执行状态，可以根据反馈修改输出值以优化控制

2. 水桶倒水：

    > 水桶高10dm，底面1dm²，一共可以装10L水

    - P控制器：

      > P = 0.5

周期 k	高度 h(k)	误差 e(k)	倒水量 u(k)	更新高度 h(k+1)
1	0.0000	10.0000	5.0000	5.0000
2	5.0000	5.0000	2.5000	7.5000
3	7.5000	2.5000	1.2500	8.7500
4	8.7500	1.2500	0.6250	9.3750
5	9.3750	0.6250	0.3125	9.6875
6	9.6875	0.3125	0.1563	9.8438

- 考虑稳态误差：

> P = 0.5

周期 k	高度 h(k)	误差 e(k)	倒水量 u(k)	更新高度 h(k+1)
1	0.0000	10.0000	5.0000	4.0000
2	4.0000	6.0000	3.0000	6.0000
3	6.0000	4.0000	2.0000	7.0000
4	7.0000	3.0000	1.5000	7.5000
5	7.5000	2.5000	1.2500	7.7500
6	7.7500	2.2500	1.1250	7.8750
7	7.8750	2.1250	1.0625	7.9375

- PI控制器：

> P = 0.5，I = 0.1

周期 k	高度 h(k)	误差 e(k)	误差累加和 ∑e(i)	倒水量 u(k)	更新高度 h(k+1)
1	0.0000	10.0000	10.0000	6.0000	5.0000
2	5.0000	5.0000	15.0000	4.0000	8.0000
3	8.0000	2.0000	17.0000	2.7000	9.7000
4	9.7000	0.3000	17.3000	1.8800	10.5800
5	10.5800	-0.5800	16.7200	1.3820	10.9620
6	10.9620	-0.9620	14.7580	0.9948	10.9568
7	10.9568	-0.9568	13.8012	0.90172	10.05509

- PID控制器：

> P = 0.5，I = 0.1，D = 0.2

周期 k	高度 h(k)	误差 e(k)	误差累加和 ∑e(i)	误差之差 e(k) - e(k-1)	倒水量 u(k)	更新高度 h(k+1)
1	0.0000	10.0000	10.0000	0.0000	6.0000	5.0000
2	5.0000	5.0000	15.0000	-5.0000	3.0000	7.0000
3	7.0000	3.0000	18.0000	-2.0000	2.9000	8.9000
4	8.9000	1.1000	19.1000	-1.9000	2.0800	9.9800
5	9.9800	0.0200	19.1200	-1.0800	1.7960	10.7760

3. 比例项（Proportional）：
    - 只含有比例项的PID输出值：out(t)=Kp∗error(t)
    - 比例项的输出值仅取决于当前时刻的误差，与历史时刻无关。当前存在误差时，比例项输出一个与误差呈正比的值，当前不存在误差时，比例项输出0。
    - Kp越大，比例项权重越大，系统响应越快，但超调也会随之增加。

- 纯比例项控制时，系统一般会存在稳态误差，Kp越大，稳态误差越小。

3. 稳态误差：

> 纯比例控制会产生稳态误差，因为控制器的输出依赖于误差本身，没有误差就没有输出，就无法维持最终的控制状态。

- PID稳态误差：系统进入稳态时，实际值和目标值存在始终一个稳定的差值。
- 稳态误差产生原因：纯比例项控制时，若误差为0，则比例项结果也为0。被控对象输入0时，一般会自发地向一个方向偏移，产生误差。产生误差后，误差非0，比例项负反馈调控输出，当调控输出力度和自发偏移力度相同时，系统达到稳态。
- 判断是否会产生稳态误差：给被控对象输入0，判断被控对象会不会自发偏移。

> 这里的自发偏移，对于电机来说，就是摩擦力；对于锅炉来说，就是环境散热；对于无人机来说，就是重力。

- 判断稳态误差的方向：给被控对象输入0，自发偏移方向即为稳态误差方向。

4. 积分项（Integral）：
    - 含有比例项和积分项的PID输出值：

> PI控制

$$out(t) = K_p * error(t) + K_i * \int_0^t error(t)dt$$

- 积分项的输出值取决于0~t所有时刻误差的积分，与历史时刻有关。积分项将历史所有时刻的误差累积，乘上积分项系数Ki后作为积分项输出值。
- 积分项用于弥补纯比例项产生的稳态误差，若系统持续产生误差，则积分项会不断累积误差，直到控制器产生动作，让稳态误差消失。
- Ki越大，积分项权重越大，稳态误差消失越快，但系统滞后性也会随之增加。

> 我们只用I项来控制时，当我们突然想让电机在一个高转速变成低转速，I项必须先把之前累积的值慢慢消除掉才行，这会导致滞后性。



而滞后性带来的问题不仅仅只是慢一点，而是会导致控制的不稳定。

5. 微分项（Differential）：
    - 含有比例项、积分项和微分项的PID输出值：

$$out(t) = K_p * error(t) + K_i * \int_0^t error(t)dt + K_d * \frac{derror(t)}{dt}$$

    - 微分项的输出值取决于当前时刻误差变化的斜率，与当前时刻附近误差变化的趋势有关。当误差急剧变化时，微分项会负反馈输出相反的作用力，阻碍误差急剧变化。
    - 斜率一定程度上反映了误差未来的变化趋势，这使得微分项具有“预测未来，提前调控”的特性。
    - 微分项给系统增加阻尼，可以有效防止系统超调，尤其是惯性比较大的系统。
    - Kd越大，微分项权重越大，系统阻尼越大，但系统卡顿现象也会随之增加。
6. PID简介：
    - PID是比例（Proportional）、积分（Integral）、微分（Differential）的缩写。
    - PID是一种闭环控制算法，它动态改变施加到被控对象的输出值（Out），使得被控对象某一物理量的实际值（Actual），能够快速、准确、稳定地跟踪到指定的目标值（Target）
    - PID是一种基于误差（Error）调控的算法，其中规定：误差=目标值-实际值，PID的任务是使误差始终为0。
    - PID对被控对象模型要求低，无需建模，即使被控对象内部运作规律不明确，PID也能进行调控。

> 对于一个自动反馈控制系统来说，有几个基本的指标。
>
> - 稳定性（P和I降低系统稳定性，D提高系统稳定性）：在平衡状态下，系统受到某个干扰后，经过一段时间其被控量可以达到某一稳定状态；
> - 准确性（P和I提高稳态精度，D无作用）：系统处于稳态时，其稳态误差（Steady-state error）；
> - 快速性（P和D提高响应速度，I降低响应速度）：系统对动态响应的要求。一般由过渡时间的长短来衡量。

7. PID公式：
    - 连续形式PID与离散形式PID公式：

> 第三种公式，当我们修改周期T时，Ki和Kd都应该要修改

- 连续形式PID：

$$out(t) = K_p * error(t) + K_i * \int_0^t error(t)dt + K_d * \frac{derror(t)}{dt}$$

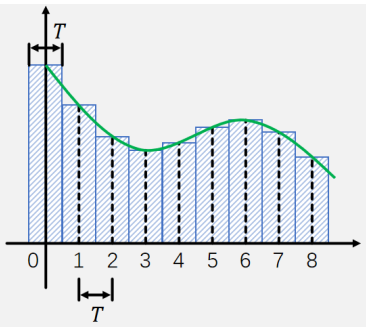
- 离散形式PID：

$$out(k) = K_p * error(k) + K_i * T \sum_{j=0}^{k} error(j) + K_d * \frac{error(k) - error(k-1)}{T}$$

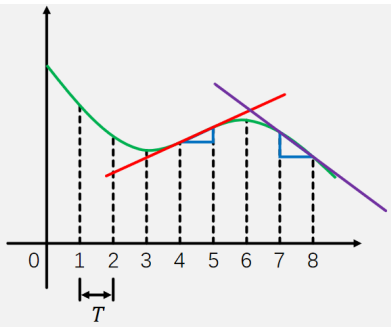
- 若将T并入K_i和K_d，则：

$$out(k) = K_p * error(k) + K_i * \sum_{j=0}^{k} error(j) + K_d * (error(k) - error(k-1))$$

> 为什么离散形式的积分要乘T，微分要除T?



$$\int_0^{8T} error(t)dt \approx \text{所有蓝色矩形的面积之和}$$
$$= T * error(0) + T * error(1) + \ldots + T * error(8)$$
$$= T \sum_{j=0}^{8} error(j)$$



$$\frac{derror(5T)}{dt} \approx \text{红色线的斜率} = \frac{error(5) - error(4)}{T}$$
$$\frac{derror(8T)}{dt} \approx \text{紫色线的斜率} = \frac{error(8) - error(7)}{T}$$

- 位置式PID与增量式PID公式：
  - 位置式PID：

$$out(k) = K_p * error(k) + K_i * \sum_{j=0}^{k} error(j) + K_d * (error(k) - error(k-1))$$

  - 当$k = k - 1$时：

$$out(k-1) = K_p * error(k-1) + K_i * \sum_{j=0}^{k-1} error(j) + K_d * (error(k-1) - error(k-2))$$

  - 两式相减，得到增量式PID：

$$\begin{aligned}\Delta out(k) = {} & K_p * (error(k) - error(k-1)) \\ & + K_i * error(k) \\ & + K_d * (error(k) - 2error(k-1) + error(k-2))\end{aligned}$$

- 位置式PID与增量式PID比较：
  - 位置式PID由连续形式PID直接离散得到，每次计算得到的是全量的输出值，可以直接给被控对象。
  - 增量式PID由位置式PID推导得到，每次计算得到的是输出值的增量，如果直接给被控对象，则需要被控对象内部有积分功能。
  - 增量式PID也可在控制器内进行积分，然后输出积分后的结果，此时增量式PID与位置式PID整体功能没有区别。
  - 位置式PID和增量式PID计算时产生的中间变量不同，如果对这些变量加以调节，可以实现不同的特性。

8. 积分限幅：
   - 要解决的问题：

如果执行器因为卡住、断电、损坏等原因不能消除误差，则误差积分会无限制加大，进而达到深度饱和状态，此时PID控制器会持续输出最大的调控力，即使后续执行器恢复正常，PID控制器在短时间内也会维持最大的调控力，直到误差积分从深度饱和状态退出。

- 积分限幅实现思路：

  对误差积分或积分项输出进行判断，如果幅值超过指定阈值，则进行限制。

  > 1. 单独对误差积分(error)进行限幅
  > 2. 对整个积分项输出(Ki*error)进行限幅

9. 积分分离：
   - 要解决的问题：

     积分项作用一般位于调控后期，用来消除持续的误差，调控前期一般误差较大且不需要积分项作用，如果此时仍然进行积分，则调控进行到后期时，积分项可能已经累积了过大的调控力，这会导致超调。

   - 积分分离实现思路：

     对误差大小进行判断，如果误差绝对值小于指定阈值，则加入积分项作用，反之，则直接将误差积分清零或不加入积分项作用。

     > 另一种说法是当误差比较小时，使用PID控制器；当误差比较大时，使用PD控制器。

10. 变速积分：
    - 要解决的问题：

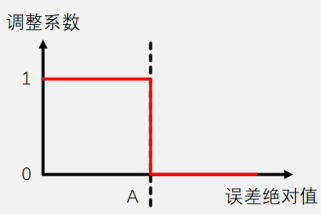
      如果积分分离阈值没有设定好，被控对象正好在阈值之外停下来，则此时控制器完全没有积分作用，误差不能消除。

      > 如果积分分离阈值（误差绝对值）设置的较大，那么当误差超过阈值之后（但不超过太多，超过太多了P项就起到作用了），I项不起作用，而P项产生的作用也不大，可能导致误差。
      >
      > 解决方法：
      >
      > 1. 减小积分分离阈值
      > 2. 变速积分

    - 变速积分实现思路：

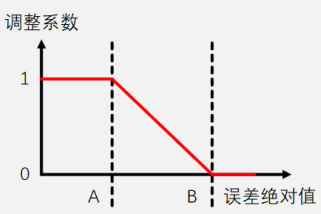
      变速积分是积分分离的升级版，变速积分需要设计一个函数值随误差绝对值增大而减小的函数，函数值作为调整系数，用于调整误差积分的速度或积分项作用的强度。

      - 积分分离

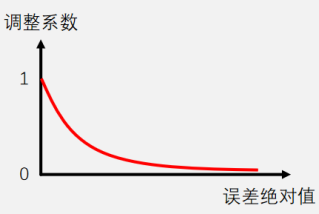
      $$y=\begin{cases}1 & (x<A)\\ 0 & (x\geq A)\end{cases}$$

      - 变速积分（线性）

      $$y=\begin{cases}1 & (x<A)\\ \dfrac{B-x}{B-A} & (A\leq x<B)\\ 0 & (x\geq B)\end{cases}$$

      - 变速积分（非线性）

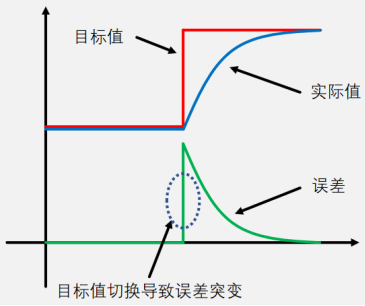
      

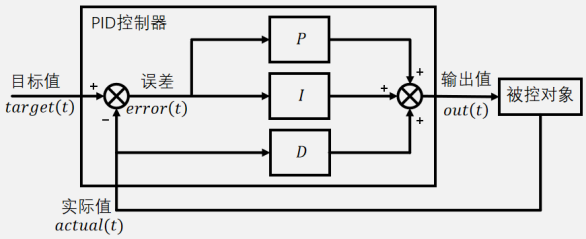
      $$y=\frac{1}{kx+1}$$

11. 微分先行：
    - 要解决的问题：

普通PID的微分项对误差进行微分，当目标值大幅度跳变时，误差也会瞬间大幅度跳变，这会导致微分项突然输出一个很大的调控力，如果系统的目标值频繁大幅度切换，则此时的微分项不利于系统稳定。

> 目标值瞬间变大时，微分会先变得特别大，然后变得特别小。
>
> D项设计的目的是为了给系统的调控增加阻尼，误差突变时，D项反而起到了帮助实际值增大的作用了。





- 微分先行实现思路：

  将对误差的微分替换为对实际值的微分。

  > 本质上D项的这个问题是因为目标值的突变（误差=目标值-实际值），那么我们只对实际值微分即可解决这个问题。
  >
  > > 为什么对实际值微分产生的效果和对误差积分一样？：
  > >
  > > 因为误差=目标值-实际值，而目标值可以看成是个常数，实际值=常数（实际值）-误差，那么对实际值微分的结果其实就是对误差微分结果的负数罢了。
  >
  > > 为什么叫微分先行？：
  > >
  > > 在用实际值计算误差之前，就直接用实际值作为输入，微分项的输入提前了。

  - 普通PID的微分项输出：

$$dout(k) = K_d * (error(k) - error(k-1))$$

  - 微分先行PID的微分项输出：

$$dout(k) = - K_d * (actual(k) - actual(k-1))$$

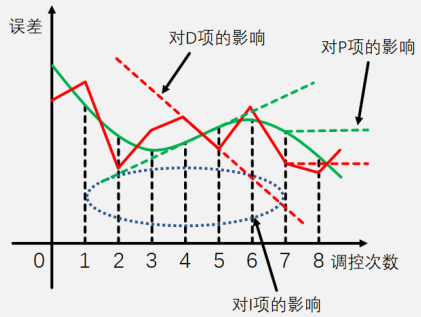
  > 微分先行的缺点：
  >
  > 加入微分先行后，PID控制会变慢。因为D项输出的很大的正向驱动力反而会让控制变化更迅速，虽然没起到D项该有的作用，但是起到了类似P项的作用。

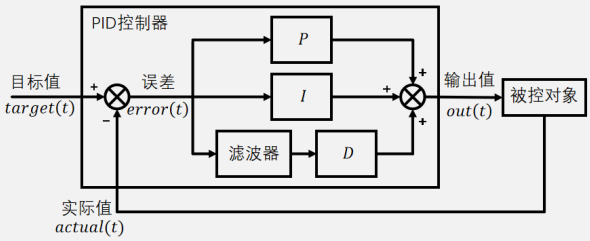
12. 不完全微分：

- 要解决的问题：

  传感器获取的实际值经常会受到噪声干扰，而PID控制器中的微分项对噪声最为敏感，这些噪声干扰可能会导致微分项输出抖动，进而影响系统性能。

  > 由图可知，噪声对误差的影响最严重。
  >
  > > 为什么只给微分项加滤波，不给传感器获取的实际值加滤波？：
  > >
  > > 可以是可以，不过可能会降低PID的响应速度（滤波器会使信号发生时延：新的信号要先与前面的值做平均，会产生滞后性）。





- 不完全微分实现思路：

  给微分项加入一阶惯性单元（低通滤波器）。

  > α参数用于调节滤波强度

  - 普通PID的微分项输出：

    $$dout(k) = K_d * (error(k) - error(k-1))$$

  - 不完全微分PID的微分项输出：

    $$dout(k) = (1-\alpha) * K_d * \big(error(k) - error(k-1)\big) + \alpha * dout(k-1)$$

13. 输出偏移：

    - 要解决的问题：

      对于一些启动需要一定力度的执行器，若输出值较小，执行器可能完全无动作，这可能会引起调控误差，同时会降低系统响应速度。

    - 输出偏移实现思路：

      若输出值为0，则正常输出0，不进行调控；若输出值非0，则给输出值加一个固定偏移，跳过执行器无动作的阶段。

      - 输出偏移的PID输出值：

        $$out(k) = \begin{cases} 0 & (out(k) = 0) \\ out(k) + offset & (out(k) > 0) \\ out(k) - offset & (out(k) < 0) \end{cases}$$

      > 但是这样会导致PID只要进行调控，系统就一直停不下来。由于实际值相对目标值很难达到正好相等的情况，所以大了就调小，小了就调大，完全停不下来。

14. 输入死区：

    - 要解决的问题：

      在某些系统中，输入的目标值或实际值有微小的噪声波动，或者系统有一定的滞后，这些情况可能会导致执行器在误差很小时频繁调控，不能最终稳定下来。

      > 解决了上面输出偏移导致PID调控停不下来的问题。

    - 输入死区实现思路：

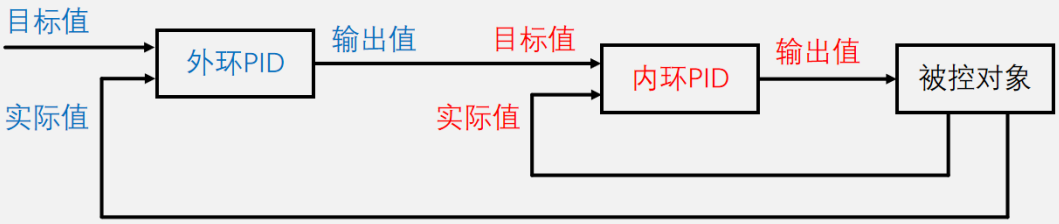
      若误差绝对值小于一个限度，则固定输出0，不进行调控。

      - 输入死区的PID输出值：

        $$out(k) = \begin{cases} 0 & (|error(k)| < A) \\ out(k) & (|error(k)| \geq A) \end{cases}$$

15. 多环串级PID：

> 单环PID只能对被控对象的一个物理量进行闭环控制，而当用户需要对被控对象的**多个维度物理量**（例如：速度、位置、角度等）进行控制时，则需要多个PID控制环路，即多环PID，多个PID串级连接，因此也称作串级PID（外环PID的输出值作用于内环PID的目标值，两个PID串级相连，因此叫做串级PID）。
>
> 多环PID相较于单环PID，功能上，可以实现对更多物理量的控制，性能上，可以使系统拥有更高的准确性、稳定性和响应速度。

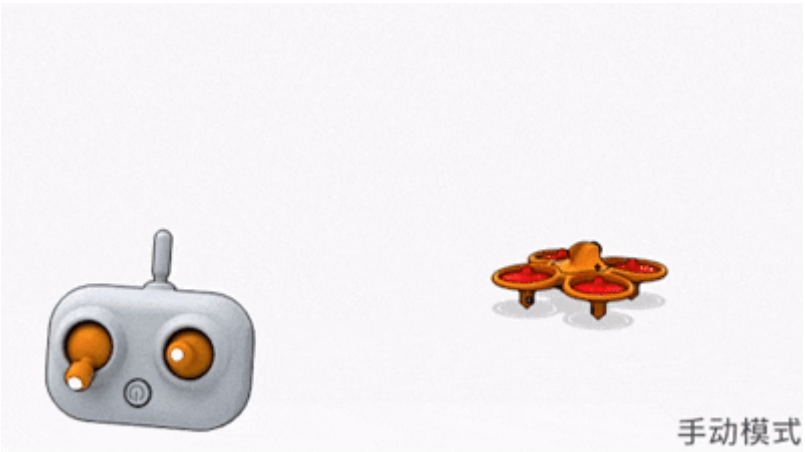


16. 遥控器摇杆和四轴飞行器实际姿态的关系：

> - 摇杆在中位：期望姿态变化率 = 0（保持当前姿态）
> - 摇杆偏转：期望姿态变化率 ∝ 摇杆偏转量
> - 摇杆最大：期望姿态变化率 = 最大允许值

- 飞行控制的两种模式：
  - 手动模式：

    > 摇杆位置 → 目标角速度
    >
    > - 特点：高手模式，完全手动控制
    > - 摇杆回中：保持当前姿态（不自动回平）
    > - 摇杆偏转：以固定角速度旋转



  - 自稳模式：

    > 摇杆位置 → 目标角度
    >
    > - 特点：新手友好，自动保持水平
    > - 摇杆回中：无人机自动回平
    > - 摇杆偏转：无人机倾斜到固定角度并保持



- 摇杆倾斜角度与无人机倾斜角度的关系：

  > 一般来说是成比例关系的

  ```
  无人机目标角度 = 摇杆偏转角度 × 无人机最大允许角度
  ```

- 飞行控制精度问题：

  > 无人机方向控制是定性的（只要向右倾斜，就能向右飞）

- 遥控器复合控制：

  > 遥控器控制的姿态角是机体坐标系下的姿态角